

Instruccion	Descripcion	Arquitectura Horizontal		Arquitectura Vertical	
		Memoria	Microinstruccion	Memoria	Microinstruccion
INV (Invert)	Cambia el signo del valor de AC, en complemento a dos	0x60	AC := ~AC	0x60	AC := ~AC
		0x61	AC := AC + 1	0x61	AC := AC + 1
		0x62	goto 0x00	0x62	goto 0x00
TABS (Top Absolute Value)	Pone en valor absoluto el valor al tope de la pila.	0x60	MAR := SP ; RD	0x60	MAR := SP ; RD
		0x61	RD	0x61	RD
		0x62	AC := MBR	0x62	AC := MBR
		0x63	ALU := AC ; if N goto 0x65	0x63	ALU := AC
		0x64	goto 0x00	0x64	if N goto 0x66
		0x65	AC := ~AC	0x65	goto 0x00
		0x66	AC := AC + 1	0x66	AC := ~AC
		0x67	MAR := SP ; MBR := AC; WR	0x67	AC := AC + 1
		0x68	WR	0x68	MAR := SP ; MBR := AC ; WR
		0x69	goto 0x00	0x69	WR
				0x6A	goto 0x00
ANDI (Indirect And)	Realiza el AND entre AC y el valor de la memoria apuntado por el operando de la instrucción. La instrucción de 16 bits tiene el código de operación en los 4 bits más significativos y el operando en los siguientes 12 bits.	0x60	TIR := band(IR, AMask)	0x60	TIR := IR
		0x61	MAR := TIR ; RD	0x61	TIR := band(TIR,AMASK)
		0x62	RD	0x62	MAR := TIR ; RD
		0x63	TIR := MBR	0x63	RD
		0x64	MAR := TIR ; RD	0x64	TIR := MBR
		0x65	RD	0x65	MAR := TIR ; RD
		0x66	AC := band(AC, MBR)	0x66	RD
		0x67	goto 0x00	0x67	TIR := MBR
				0x68	AC := band(AC,TIR)
				0x69	goto 0x00
MULD (Direct Mult)	Multiplica el valor del operando por AC y deja el resultado en AC	0x60	TIR := band(IR, AMask)	0x60	TIR := IR
		0x61	MAR := TIR ; RD	0x61	TIR := band(TIR, AMask)
		0x62	RD	0x62	MAR := TIR ; RD
		0x63	TIR := MBR	0x63	RD

		0x64	IR := AC	0x64	TIR := MBR
		0x65	AC := 0	0x65	IR := AC
		0x66	ALU := TIR ; if N then goto 0x6B	0x66	AC := 0
		0x67	ALU := TIR ; if Z then goto 0x70	0x67	ALU := TIR
		0x68	AC := AC + IR	0x68	if N then goto 0x6E
		0x69	TIR := TIR + (-1)	0x69	ALU := TIR
		0x6A	goto 0x67	0x6A	if Z then goto 0x73
		0x6B	TIR := inv(TIR)	0x6B	AC := AC + IR
		0x6C	TIR := TIR + 1	0x6C	TIR := TIR + (-1)
		0x6D	IR := inv(IR)	0x6D	goto 0x69
		0x6E	IR := IR + 1	0x6E	TIR := inv(TIR)
		0x6F	goto 0x67	0x6F	TIR := TIR + 1
		0x70	goto 0x00	0x70	IR := inv(IR)
				0x71	IR := IR + 1
				0x72	goto 0x69
				0x73	goto 0x00
DIVI (Indirect Div)	Divide AC por el valor en la memoria apuntado por el operando (12 bits) y deja el resultado de la división en AC y el resto en TIR.	0x60	TIR := band(IR, AMask)	0x60	TIR := IR
		0x61	MAR := TIR ; RD	0x61	TIR := band(TIR, AMask)
		0x62	RD	0x62	MAR := TIR ; RD
		0x63	TIR := MBR	0x63	RD
		0x64	MAR := TIR ; RD	0x64	TIR := MBR
		0x65	RD	0x65	MAR := TIR ; RD
		0x66	TIR := MBR	0x66	RD
		0x67	IR := AC	0x67	TIR := MBR
		0x68	AC := 0	0x68	IR := AC
		0x69	TIR := inv(TIR)	0x69	AC := 0
		0x6A	TIR := TIR + 1	0x6A	TIR := inv(TIR)
		0x6B	IR := IR + TIR	0x6B	TIR := TIR + 1
		0x6C	ALU := IR ; if N then goto 0x6F	0x6C	IR := IR + TIR
		0x6D	AC := AC + 1	0x6D	ALU := IR
		0x6E	goto 0x6B	0x6E	if N then goto 0x71
		0x6F	TIR := inv(TIR)	0x6F	AC := AC + 1

		0x70	TIR := TIR + 1	0x70	goto 0x6C
		0x71	IR := IR + TIR	0x71	TIR := inv(TIR)
		0x72	TIR := IR	0x72	TIR := TIR + 1
		0x73	goto 0x00	0x73	IR := IR + TIR
				0x74	TIR := IR
				0x75	goto 0x00
JTE (Jump Top Equal)	Salta a la dirección del operando, si el tope de la pila es igual a AC.	0x60	MAR := SP ; RD	0x60	TIR := AC
		0x61	RD	0x61	MAR := SP ; RD
		0x62	TIR := inv(AC)	0x62	RD
		0x63	TIR := TIR + 1	0x63	AC := MBR
		0x64	TIR := MBR + TIR	0x64	TIR := inv(TIR)
		0x65	ALU := TIR ; if Z then goto 0x67	0x65	TIR := TIR + 1
		0x66	goto 0x00	0x66	AC := AC + TIR
		0x67	TIR := band(IR, AMask)	0x67	TEST AC
		0x68	PC := TIR	0x68	if Z goto 0x6D
		0x69	goto 0x00	0x69	TIR := inv(TIR)
				0x6A	TIR := TIR + 1
				0x6B	AC := TIR
				0x6C	goto 0x00
				0x6D	TIR := inv(TIR)
				0x6E	TIR := TIR + 1
				0x6F	AC := TIR
				0x70	TIR := IR
				0x71	TIR := band(TIR, AMask)
				0x72	PC := TIR
				0x73	goto 0x00
BSR (Bit Scan Reverse)	Busca el bit 1 menos significativo de la celda de memoria que se encuentra al tope de la pila y devuelve su índice en AC. En caso que el tope de la pila contenga 0x0000, deja en AC el valor -1.	0x60	MAR := SP ; RD	0x60	MAR := SP ; RD
		0x61	RD	0x61	RD
		0x62	TIR := MBR	0x62	TIR := MBR
		0x63	ALU := TIR ; if Z then goto 0x6B	0x63	ALU := TIR
		0x64	AC := 0	0x64	if Z then goto 0x6E

		0x65	IR := band(TIR, +1)	0x65	AC := 0
		0x66	ALU := IR ; if Z then goto 0x68	0x66	IR := TIR
		0x67	goto 0x00	0x67	IR := band(IR, +1)
		0x68	TIR := rshift(TIR)	0x68	ALU := IR
		0x69	AC := AC + 1	0x69	if Z then goto 0x6B
		0x6A	goto 0x65	0x6A	goto 0x00
		0x6B	AC := -1	0x6B	TIR := rshift(TIR)
		0x6C	goto 0x00	0x6C	AC := AC + 1
				0x6D	goto 0x66
				0x6E	AC := -1
				0x6F	goto 0x00
BSF (Bit Scan Forward)	Lee el valor apuntado por su operando (12 bits) y guarda el índice del primer "1" en AC. En caso de ser cero, deja -1 en AC.	0x60	TIR := band(IR, AMask)	0x60	TIR := IR
		0x61	MAR := TIR ; RD	0x61	TIR := band(TIR, AMask)
		0x62	RD	0x62	MAR := TIR ; RD
		0x63	TIR := MBR	0x63	RD
		0x64	ALU := TIR ; if Z then goto 0x6C	0x64	TIR := MBR
		0x65	AC := 0	0x65	ALU := TIR
		0x66	IR := band(TIR, +1)	0x66	if Z then goto 0x70
		0x67	ALU := IR ; if Z then goto 0x69	0x67	AC := 0
		0x68	goto 0x00	0x68	IR := TIR
		0x69	TIR := rshift(TIR)	0x69	IR := band(IR, +1)
		0x6A	AC := AC + 1	0x6A	ALU := IR
		0x6B	goto 0x66	0x6B	if Z then goto 0x6D
		0x6C	AC := -1	0x6C	goto 0x00
		0x6D	goto 0x00	0x6D	TIR := rshift(TIR)
				0x6E	AC := AC + 1
				0x6F	goto 0x68
				0x70	AC := -1
				0x71	goto 0x00
	Apaga (pone en 0) el bit 1 menos significativo, dejando el resto intacto, de la salida de memoria en la posición de	0x60	TIR := band(IR, AMask)	0x60	TIR := IR
		0x61	MAR := TIR ; RD	0x61	TIR := band(TIR, AMask)

BLSR (Bitwise Reset)	la celda de memoria en la posición de memoria indicada en su único operando (12 bits). Es el equivalente por hardware a hacer $x = x \& (x - 1)$ en C.	0x62	RD	0x62	MAR := TIR ; RD
		0x63	IR := TIR	0x63	RD
		0x64	AC := MBR	0x64	IR := TIR
		0x65	TIR := AC	0x65	AC := MBR
		0x66	AC := AC + (-1)	0x66	TIR := AC
		0x67	AC := band(AC, TIR)	0x67	AC := AC + (-1)
		0x68	MAR := IR ; MBR := AC ; WR	0x68	AC := band(AC, TIR)
		0x69	WR	0x69	MAR := IR ; MBR := AC ; WR
		0x6A	goto 0x00	0x6A	WR
				0x6B	goto 0x00
POPCNT (Population Count)	Cuenta el número de bits establecidos en 1 (bits activos) dentro del registro AC y lo guarda en la posición de memoria indicada en su único operando (12 bits).	0x60	TIR := band(IR, AMask)	0x60	TIR := IR
		0x61	IR := TIR	0x61	TIR := band(TIR, AMask)
		0x62	TIR := AC	0x62	IR := TIR
		0x63	AC := 0	0x63	TIR := AC
		0x64	ALU := TIR ; if Z then goto 0x6B	0x64	AC := 0
		0x65	ALU := TIR ; if N then goto 0x68	0x65	ALU := TIR
		0x66	TIR := lshift(TIR)	0x66	if Z then goto 0x6E
		0x67	goto 0x64	0x67	ALU := TIR
		0x68	AC := AC + 1	0x68	if N then goto 0x6B
		0x69	TIR := lshift(TIR)	0x69	TIR := lshift(TIR)
		0x6A	goto 0x64	0x6A	goto 0x65
		0x6B	MAR := IR ; MBR := AC ; WR	0x6B	AC := AC + 1
		0x6C	WR	0x6C	TIR := lshift(TIR)
		0x6D	goto 0x00	0x6D	goto 0x65
				0x6E	MAR := IR ; MBR := AC ; WR
				0x6F	WR
				0x70	goto 0x00